
WHITE PAPER · WORK, ADOPTION & JUDGMENT

The Coordination Tax

Map the work between tasks before claiming the savings

Marco Policani

Enterprise Portfolio · PMO · AI Operating Governance

policani.net · linkedin.com/in/marcpolicani

July 2026

EXECUTIVE SUMMARY

Automating a task can push work into the handoffs, checks, and exceptions around it; map and design that work before counting the savings.

Automating a task can shrink it while the work between tasks (routing, translating, waiting, approving, reconciling, repairing) quietly absorbs the savings. Mapping that surrounding work one workflow at a time shows where the savings went: trace real cases end to end, sort each handoff and check by whether it earns its place, price the added work at the seniority of the people who do it, and count freed capacity only when the whole workflow shows it.

- 01** Trace real cases, not the process diagram: much of the hidden work sits in steps people added to make the official process function, and diagrams rarely show them.
- 02** Before cutting any step that exists to coordinate work, such as a review or handoff, ask what it produces; some reviews and handoff conversations are where defects get caught, and removing them makes failures more expensive.
- 03** Count freed capacity only after the workflow has run long enough to settle, usually one to two quarters, or you will staff to savings that never arrived.

The savings that never showed up

Automation can cut the effort a task takes and still leave the work around it untouched. It can even add to that work. Someone still has to route the job, reconcile the numbers, approve the output, explain it, and fix what breaks. Leaders should map that work and measure the whole workflow before claiming a productivity gain or giving the tool more independence.

One automation review made this vivid for me. By its own numbers, a report-drafting workflow had succeeded completely. Drafting time per report fell from four hours to twenty minutes. The team was proud and the tooling was good. The function's total throughput had not improved at all. We traced a month of reports to find where the fourteen weekly hours of savings had gone. The answer was everywhere and nowhere. Analysts now spent time picking which source data the tool should use.

They fixed its formatting for the downstream application. They explained in review meetings which parts were generated. They waited on an approval step that had been added because the reports were generated. They also reconciled gaps between the tool's numbers and the ledger's. The task itself had collapsed. The work around it had grown and spread.

That is the coordination tax: the work between the tasks. Routing, translation, waiting, approval, reconciliation, explanation, repair. It rarely shows up in an automation business case, because it is spread across many roles, buried in calendars and inboxes, and owned by no one in particular. Task automation puts its benefit in one visible place and scatters its costs across many invisible ones. Unless someone maps and measures that coordination, the business case weighs a real saving against an expense nobody has priced. Researchers have a formal definition worth borrowing. Coordination is the managing of dependencies among activities: shared resources, handoffs from producer to consumer, sequencing constraints. Each of those is real work, whether or not any ledger names it.[3]

This paper offers the map as the instrument. Call it a coordination map: a six-part review of one workflow, run before any decision to scale. Model-based research on when coordination between tasks is genuinely avoidable offers a useful hypothesis about how AI may change work.[1] NIST's risk framework supplies the more durable governance frame.[2] Neither one replaces measuring your own workflow, which is the whole method here. I put it together by tracing where my own automations' savings had gone. It is written for operating executives, transformation leaders, workflow owners, and AI program leaders.

Why the tax stays invisible

Coordination work has three traits that keep it off the books. It is broken into fragments: thirty seconds of routing here, five minutes of reconciliation there, no piece big enough to name. It is spread out: each role carries a sliver, so no one person feels the total. Finally, it stretches: capable people absorb it into their days without complaint, so it reads as busyness rather than cost. Work with those three traits does not show up in time studies built around tasks. Automation cases are built around tasks. The size of this hidden load has been measured on its own. Cross, Rebele, and Grant collected data over two decades. In that data, the time managers and employees spend in collaborative activities rose by 50 percent or more, and at many companies meetings, calls, and email take around 80 percent of people's time. [6] A task-based automation case never looks into that pool.

Automation then adds work of its own. Someone has to check the generated output against sources, put it in the format the downstream application needs, and explain it

to the people who have to know what to trust. New approval steps appear precisely because the work is machine-made. More odd cases reach people at the edges of what the tool can handle. None of this is unusual. It is what normally happens when you drop a fast, narrow tool into a broad, human process. The question is never whether the coordination changes. It is whether anyone is watching it change.

The risk adds up across a portfolio. Ten automations, each a local win by task metrics, can add up to a function that is no faster, more fragile, and harder to staff. That happens because each one quietly pulled the same experienced people into checking, translating, and repairing roles that nobody designed or counted. The portfolio celebrates ten wins and wonders why the outcome curve is flat. The answer sits in the work between the tasks, which no one mapped. This mismeasurement is not unique to one function. Brynjolfsson, Rock, and Syverson show that when firms adopt a general-purpose technology, measured productivity growth is understated at first, because resources go into complementary intangible investments the statistics do not capture. They call it the productivity J-curve.[7] The same thing happens inside a portfolio of automations. The task saving books at once. The coordination work that comes with it stays uncouned. The outcome curve stays flat until someone prices it.

Agentic automation raises the stakes here rather than escaping them. An agent that plans, calls tools, and hands work to other agents is a small coordination structure of its own. It has the same handoff contracts, the same odd cases at the edges, and the same translation layers. It runs them at machine speed, without the capable people who used to absorb design gaps quietly. Organizations that mapped the coordination among their own people will recognize every failure mode. The ones that never learned to see coordination will meet it at a much less forgiving pace. Software has known this failure for years. Coordination problems there have been identified as clashes between how the code is structured and how the team working on it is organized.[4]

The coordination map

The map is a review with six disciplines, applied to one workflow at a time. It studies the path rather than the task, and it puts the results into the same business case that proposed the automation. Nothing in it needs new tooling. It needs observation, which is cheaper and rarer.

EXHIBIT 1

Six disciplines expose where automation's savings actually went

- 1 **Map work between tasks**
What you study is the whole path across people, applications, decisions, and queues, not the single activity the tool performs.
- 2 **Tell useful coordination from waste**
Some interaction creates judgment, learning, trust, safety, or necessary integration; eliminating it can degrade the outcome.
- 3 **Find the work around the tool**
Automation often creates new translation between generated output and the applications or people that must use it.
- 4 **Design handoffs deliberately**
Every transfer needs a clear object, acceptance test, owner, and response expectation.
- 5 **Own the cases the tool cannot handle**
A faster happy path can increase the absolute number or complexity of cases requiring human coordination.
- 6 **Measure the whole path**
Value should appear in end-to-end cycle time, quality, capacity, coverage, decision speed, or risk; tool usage and task duration are supporting measures.

Map work between tasks

The first discipline traces real cases across people, applications, decisions, and queues, from the moment the demand arrives to the moment the outcome is delivered. Mark every transfer, wait, approval, dependency, and backward loop. Trace the cases you observe, not the process diagram. The diagram shows the intended path. The tax lives on the actual one, in the steps people quietly added so the intended path would survive contact with reality. Almost by definition, no document mentions those steps and no owner defends them.

The evidence is a touch-and-queue ledger for each case: how many hands, how much waiting between hands, how much rework, which role carried each step, and why cases moved backward when they did. A dozen traced cases usually show where the time goes. The rule that follows: make no automation decision until the proposal can point at this map and say which links it removes, which it moves, and which it creates. An answer like "It makes the drafting step faster" covers one point on the map. The tax sits on the lines between the points.

Here is what tracing actually looks like. Pick twelve recent cases, some routine and some messy. For each one, interview the people who touched it. Ask about this case, not about the process. When did it arrive on your desk? What did you need before you could act? Whom did you wait on? What did you send back, and why? Plot the twelve journeys on one wall. For most cases, the wall will disagree with the process diagram in the same three or four places. Those disagreements are the shadow steps the organization built because the official path did not work.

Tell useful coordination from waste

The second discipline resists the efficiency reflex. Some interaction is how judgment forms, defects come to light, novices learn, and cross-functional risk gets caught. It is the review that catches the mispriced contract, the handoff talk where the next team hears the case's history, the escalation that pulls a senior eye onto something unusual. Delete those links and you get a faster workflow that fails more expensively.

The test for each coordination point is what it produces: a decision, a defect caught, knowledge passed on, a control, a relationship the work later depends on, or nothing but motion. Honest answers sort the links into four bins. Valuable: keep it, maybe strengthen it. Necessary but reducible: same job, lighter mechanism. Avoidable: a route or a rule can remove it. Created by the change itself: a cost of the automation, to be counted as one. The bins matter because automation vendors and efficiency programs tend to treat the whole category as waste. Organizations that believe them delete the very checks that were catching their mistakes.

Binning takes judgment. In an order-exception workflow I reviewed, a daily cross-team standup looked like a textbook deletion candidate: thirty minutes, eight people, mostly status. Traced against real cases, it turned out to be the only point where pricing learned about inventory substitutions before customers did. Deleting it would have turned a coordination cost into a stream of refunds. The standup stayed, shrank to twelve minutes, and gained an agenda. Meanwhile a weekly "alignment review" that everyone defended traced to zero decisions and zero catches across a month of cases. Nobody missed it when it ended. The bins come out of the cases, not the org chart.

Find the work around the tool

The third discipline hunts the newest form of the tax: the work of translating between generated output and the applications and people that must use it. Users clean up formats. They re-key content into the official record. They check claims against

sources. They note for later readers where the content came from. They also prepare inputs the tool can handle. This work is nearly always invisible. It happens inside the users' existing roles, measured in minutes per case, at exactly the point the task metric does not cover. Machine-learning systems show a technical parallel. They build up hidden coupling, meaning parts that quietly depend on each other, which commonly brings massive ongoing maintenance costs.[5]

The method is observation. Sit with real users. Time the work they do before and after the tool runs. Sort it into categories: preparing inputs, correcting output, translating formats, checking claims, explaining results. The findings convert straight into design work. If translation dominates, that points to interface or integration work. If checking dominates, to provenance features or better data upstream. If explaining dominates, to output packaged differently for later readers. Work around the tool that cannot be designed away belongs in the business case as a permanent operating cost. Price it at the seniority of whoever does it, which is often uncomfortably high.

The work around the tool also lands unevenly, which compounds its cost. The checking and translation fall disproportionately on experienced staff, because checking is what takes judgment. So the automation's hidden payroll is drawn from exactly the people the organization can least spare, while the visible savings were priced at the average. A ledger that records who does those minutes, and not just how many, regularly flips a business case from positive to negative.

Design handoffs deliberately

The fourth discipline engineers the transfers that remain. A handoff is a small contract. It names an object with its evidence attached, a test the receiver uses to accept it, an owner on each side, and the response expected. Automation strains these contracts by changing volume and variety at machine speed. A sped-up upstream step can flood a downstream queue that lacks the context, the capacity, or the authority to act on what arrives.

A handoff example from the same order-exception review: the automated classifier pushed case intake to three times its old rate, into a resolution team whose acceptance criteria had never been written down. Cases arrived missing the two fields resolvers actually needed. Clarification requests tripled. End-to-end time for the "faster" workflow got worse. The fix was a handoff contract: two required context fields, an acceptance check at the mouth of the queue, and an intake rate matched to resolution capacity. None of it touched the classifier. The automation was fine. What failed was the edge nobody had designed.

The evidence is queue telemetry at each transfer: age, acceptance rate, clarification requests, incomplete handoffs sent back, and downstream teams swinging between idle and overloaded. The design responses are unglamorous and effective. Attach the context the next team actually needs. Set acceptance criteria so rejection is cheap and early. Throttle upstream production to match what downstream can absorb. That last one is culturally hard. Slowing an automated step feels like waste. When a queue grows faster than the team drawing from it, though, the extra work just sits there going stale. That is not throughput.

Own the cases the tool cannot handle

The fifth discipline prices the cases that fall outside what the automation can handle. Call them exceptions. A faster happy path, the cases the automation handles cleanly, almost always raises the absolute number of exceptions that reach people. Volume grows, and odd cases gather at the automation's edges. The coordination cost is specific. Exceptions arrive stripped of context. They bounce between teams looking for an owner. They also consume the scarcest people, because only the experienced can handle what the machine could not.

Two numbers matter most to the review, and both need honest bookkeeping. First, the specialist hours each exception consumes. Count the tour through teams to find an owner, and the time spent rebuilding context before anyone could act, not just the fix itself. Second, the ratio of that consumption to the hours the automation saves on routine cases. That ratio is the automation's real margin, and it degrades quietly as volume grows. The rule: hold back on scale when demand from those cases approaches the capacity the happy path releases. Designing the exception path well is its own discipline, deeper than this paper goes. The point here is narrower: the exception path is part of the coordination work, and its cost belongs in the same ledger.

One caution comes from every queue discipline. Measure what the next team can really absorb, not what their stated capacity says. A downstream team "absorbing" the faster flow by triaging quicker and reading less deeply has not absorbed anything. It has traded a visible queue for invisible quality debt. Sampling downstream decision quality after an upstream speed-up is the check that catches this, and it belongs in the pilot design rather than the post-incident review.

Measure the whole path

The sixth discipline moves the success metric from the task to the path. Value shows up, or does not, in end-to-end cycle time, total touches, outcome quality, capacity actually released for other work, coverage expanded, decisions made faster, or risk reduced. Tool usage, task duration, and adoption are supporting indicators. They measure activity at one point, while the claim being tested is about the whole path.

The method is a disciplined before-and-after against the traced baseline: the same case types, the same path measures, and output quality sampled by people who know the work well enough to judge it. The report-drafting story that opened this paper is the outcome to expect when the work around the tool and the handoffs get skipped. Task time collapses, path time holds, and the difference is the tax. Scale when the path improves and the remaining coordination is visible and owned. Redesign when the path is flat. Treat a flat path with a celebrated task metric as the warning sign it is.

It helps to hold the research at the right distance. Coordination analysis of the kind Ju formalizes asks when the work between activities is itself removable, as distinct from whether the activities can be automated.[1] That is a hypothesis about categories of work in general. A coordination map is the empirical instrument for one workflow in particular. Every workflow I have traced has surprised its own managers somewhere. That is the practical argument for measuring rather than modeling when the decision is local.

Using the map to decide

The six disciplines run as one review, and the output fits on a wall: the traced path, links colored by bin, work around the tool counted, handoff contracts marked, exception costs attached, and the path-level baseline next to the proposal's claims. Decision-makers can then see what a task-level case never shows them: what the automation actually buys, at what coordination price, and where design work would change the answer. People who would dispute a spreadsheet will accept a traced case they can follow with a finger.

EXHIBIT 2

The scale decision each discipline informs

Map work between tasks

Automate only after leaders can see whether the proposed change removes or relocates coordination.

Tell useful coordination from waste

Keep valuable coordination, redesign avoidable routing, and automate only the stable remainder.

Find the work around the tool

Include that work in the business case, and redesign the interface or process when it dominates.

Design handoffs deliberately

Throttle or reshape upstream automation when the receiving team cannot absorb the volume.

Own the cases the tool cannot handle

Constrain scale when recovery demand exceeds the capacity saved on ordinary cases.

Measure the whole path

Scale when the workflow result improves and the remaining coordination is both visible and owned.

The map's second use is sequencing. Most workflows hold several automation candidates, and the map shows which order pays best. Automating a step whose downstream queue is already backed up just builds a bigger pile. Fixing the translation work first buys room for everything after it. A review that says "not yet, and here is what to do first" is the map earning its keep. The alternative was learning the sequence from the failure.

The costliest single error the map prevents is declaring capacity released before the path confirms it. Business cases claim the analyst hours the task no longer needs. Budgets absorb the claim. In reality, the hours went to work around the tool and to exceptions nobody counted. The function is then staffed for savings that never arrived, and it runs hot for a year. Counting released capacity at the path level, after the coordination has settled, is what keeps automation programs solvent.

Making the map a habit

The review is light to install:

- Map one high-volume workflow from cases you observe, not from process documentation alone.
- Label every coordination point as valuable, necessary but reducible, avoidable, or created by the change itself.
- Pilot changes where the tool meets people and systems, and on the exception path, not just on the task.
- Review end-to-end measures and specialist load before expanding volume or authority.

The review also needs a sponsor who is ready for what it finds, because these findings implicate design choices that have authors. The approval step that traces to zero catches was somebody's governance win. The shadow reconciliation belongs to a team that built it in self-defense. Present the findings as how the process behaves rather than as individual fault — the discipline incident reviews learned decades ago — and the second review stays welcome. A map that becomes a blame instrument gets no more honest cases to trace.

The first review takes about two weeks of part-time effort for a substantial workflow: a dozen traced cases, a few observation sessions, one wall. Run it on a workflow that already has an automation proposed, so the findings land in a live decision rather than a methodology showcase. The second review is faster. By the fourth, tracing is a habit the analysts apply without being asked, which is the real installation.

Ownership belongs to the workflow owner. The automation or AI program function supplies the method and reads across workflows for what repeats. Work around the tool clustering at the same downstream application across several reviews is a case for investing in integration. Exception costs clustering around the same class of case is a case for fixing data or policy upstream. Individual reviews find taxes. The portfolio view finds what keeps producing them.

The six records to keep

A mapping practice leaves evidence:

- Traced paths with touch, queue, and rework baselines, before and after each change.

- Coordination links sorted into the four bins, with the created-by-the-change bin priced.
- Minutes of work around the tool per case, by category and by the seniority doing it.
- Handoff queues inside their design tolerances, and throttles that fired when they were not.
- Exception hours against happy-path savings, tracked as volume grows.
- Path-level outcomes — cycle time, quality, released capacity — that confirm or refute each decision to scale.

Timing the measurement window honestly matters as much as picking the measures. The weeks right after an automation lands are the least representative of its life. Novelty effort looks like work around the tool, and learning-curve slowdown looks like permanent cost. Measure the path once the coordination has settled, usually one to two quarters in. Then keep the baseline alive, because coordination grows back around any tool as the workflow's cases and people change.

The portfolio number worth an executive's attention is the coordination created by the changes themselves, summed across all automations and trended quarterly. Flat or falling while automation grows means the design discipline is working. Rising faster than automation value means the program is buying task speed with path capacity. The trade ends with the coordination tax consuming the program.

Three objections

Coordination is not inherently waste. Complex work needs consultation and cross-functional judgment. The tax is coordination whose purpose is unclear, whose route is avoidable, or whose cost was shifted out of view. Redesign should preserve necessary collaboration.

Nor is the map an argument against ambitious automation. Its verdicts regularly run the other way. A workflow whose coordination is mostly avoidable routing and re-keying is a candidate for deeper automation than anyone proposed, because the map shows the interaction would simplify rather than strain. The map is neutral. It removes blindness in both directions and leaves ambition intact.

The objection from speed says that mapping slows an automation program while competitors move faster. It undercounts what the map prevents. A program that skips the review ships more automations per quarter and takes on unpriced coordination debt in every one. The debt compounds into the flat outcome curve and the burned-out specialist layer that stall such programs in year two. Two weeks of tracing per

workflow is the carrying cost of knowing what the automation actually does. It is the fastest speed a program can sustain.

When to call an automation finished

The change is from automating tasks to redesigning paths. The task is one point on the map. The value sits on the lines between the points. The map is what separates automation that compounds from automation that merely moves work out of sight. It is drawn from observed cases, sorted by what each link produces, priced at the seniority that carries it, and owned by someone who will keep the baseline alive.

Leaders do not need to run the reviews themselves. They need to change what counts as done. An automation is complete when the path improves, the remaining coordination has owners, and the released capacity survives a path-level audit. Holding that definition against vendors, against internal enthusiasm, and against their own task-metric dashboards is the executive contribution. It is enough.

The report-drafting team did eventually get its fourteen hours, eight months later. An integration ended the re-keying. A provenance banner ended most of the explaining. The added approval step was retired on evidence of its own uselessness. The tool never changed. The work around it did.

Notes and sources

- [1] Harang Ju, "When Coordination Is Avoidable: A Monotonicity Analysis of Organizational Tasks," 2026 preprint. <https://arxiv.org/abs/2602.18673>
- [2] NIST, "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile," July 2024. <https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.600-1.pdf>
- [3] Thomas W. Malone and Kevin Crowston, "The Interdisciplinary Study of Coordination," ACM Computing Surveys 26, no. 1 (1994): 87–119. <https://crowston.syr.edu/sites/default/files/acmcs94.pdf>
- [4] Chintan Amrit, Jos van Hillegersberg, and Kuldeep Kumar, "Identifying Coordination Problems in Software Development: Finding Mismatches between Software and Project Team Structures," 2012. <https://arxiv.org/abs/1201.4142>
- [5] D. Sculley, Gary Holt, Daniel Golovin, et al., "Hidden Technical Debt in Machine Learning Systems," Advances in Neural Information Processing Systems 28 (NeurIPS 2015).

<https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf>

[6] Rob Cross, Reb Rebele, and Adam Grant, "Collaborative Overload," Harvard Business Review, January–February 2016. <https://hbr.org/2016/01/collaborative-overload>

[7] Erik Brynjolfsson, Daniel Rock, and Chad Syverson, "The Productivity J-Curve: How Intangibles Complement General Purpose Technologies," NBER Working Paper 25148 (2018); published in American Economic Journal: Macroeconomics 13(1), 2021. https://www.nber.org/system/files/working_papers/w25148/w25148.pdf

About the author

Marco Policani is an enterprise portfolio, PMO, and AI operating-governance leader. He builds portfolio governance systems where AI carries the analytical load and named humans own every decision — an approach documented across case studies, walkthroughs, and working governance guides on his portfolio site.

Portfolio & governance library: policani.net/governance · Full portfolio: policani.net · LinkedIn: linkedin.com/in/marcpolicani

This white paper may be shared freely with attribution. © 2026 Marco Policani.